

Then we can use dependency injection to inject the `OrdinalPipe` into the constructor, which makes it available privately as `this.pipe`, where we can call `this.pipe.transform()`:

```
• export class App {
•   constructor(private pipe: OrdinalPipe) {
•     let numbers = [
•       1,2,3,4,5,6,7,8,9,10,
•       11,12,13,14,15,16,17,18,19,20
•     ];
•     this.numbers = numbers.map(number => this.pipe.
transform(number));
•   }
• }
```

Although pipes are not new to angular (Angular 1.x calls them filters) they are very useful in removing both boilerplate code as well as using functions as simple operations to be performed on large datasets.

Useful Links

- <http://ngmigrate.telerik.com/custom-pipes-angular-2#creating-a-custom-pipe>
- <http://ngmigrate.telerik.com/custom-pipes-angular-2#using-pipes-in-templates>
- <http://ngmigrate.telerik.com/custom-pipes-angular-2#passing-arguments-to-pipes>
- <http://ngmigrate.telerik.com/custom-pipes-angular-2#filtering-in-component-classes-with-pipes> 